Introduzione

Durante l'attività di testing si è cercato di garantire una copertura ottimale e attivare tutte le istruzioni, archi e predicati principali. Per fare ciò sono state applicate le seguenti tecniche:

- **Partizionamento d'Equivalenza**
- **Analisi dei Valori Limite**
- **State Transition Testing**
- **Error Guessing e Defensive Programming**

ServizioReteTCP e RichiestaSH

Per `ServizioReteTCP`, per evitare che il test dipendesse da cause esterne potenzialmente invalidanti, è stato istanziato un server locale ad hoc capace di rispondere con JSON preimpostati. Di seguito sono riportati i test cases individuati.

- **TC-01:** Validazione formato indirizzo di rete. In caso di formato non valido è previsto il lancio di eccezioni opportune.
- **TC-02:** Ricezione messaggio errore dal dispositivo. Una eccezione opportuna deve essere sollevata.
- **TC-03:** Nessun server in ascolto. La connessione fallisce. Una eccezione viene lanciata.
- **TC-04: Happy path** Invio comando e verifica che il metodo di parsing scomponga correttamente la risposta.
- **RichiestaSH:**
  - Verificata la robustezza del costruttore e la corretta formattazione della stringa JSON finale.

RegistroTargets

Si è ritenuto inserire un controllo nel metodo `addTarget()` per evitare che i target venissero sovrascritti o duplicati.
La classe è stata testata grazie all'ausilio di una classe anonima che istanzia e ridefinisce la classe `Target`.

- **TC-01** `addTarget()` **(Happy path):** Verifica aggiunta targets.
- **TC-02** `addTarget()` **:** Il Target è null: `IllegalArgumentException` deve essere lanciata.
- **TC-03** `addTarget()` **:** L'id del target è già presente nel registro. L'inserimento viene bloccato.
- **TC-04** `removeTarget()` **(Happy path):** Verifica rimozione target.
- **TC-05** `removeTarget()` **:** Rimozione target con id inesistente. Non succede nulla.
- **TC-06** `getTarget()` **:** Se richiedo un target con id nullo o inesistente il metodo deve restituire `null`
- **TC-07** `factoryReset()` **:** Test factory reset. Il registro deve essere nullo dopo il reset.

Target

La classe è stata testata grazie all'ausilio di una classe anonima che istanzia e ridefinisce la classe `Target`.

- **TC-01** `addTarget()` **(Happy path):** Test costruttore. Id valido.
- **TC-02/03** `addTarget()` **:** Test costruttore. Id nullo o vuoto. Una `IllegalArgumentException` deve essere lanciata.

Dispositivo

Durante l'attività di testing della suddetta classe è emersa una falla nella progettazione del metodo `aggiornaStato()`. Era prima permesso l'inserimento di nuovi parametri allo stato dei dispositivi. Ciò non deve essere possibile! Il sistema deve ottenere le informazioni circa lo stato dai dispositivi stessi. Il controllo è stato aggiunto. Per supportare il controllo è stato inoltre modificato il costruttore in modo da inizializzare lo stato dei

dispositivi con valore `"SCONOSCIUTO"`. Si prevede in futuro che i dispositivi aggiornino il proprio stato all'avvio oltre che in risposta alle transizioni.
I seguenti test si concentrano sul comportamento strutturale e sulle transizioni di stato.

- **TC-01** `getStato()`**:** Verifica comportamento copia difensiva stato. L'istanza originale della mappa rappresentante lo stato non deve essere condivisa. Viene restituita una copia. A fronte di modifiche alla copia lo stato interno rimane inalterato.
- **TC-02** `aggiornaStato()`**:** Primo inserimento dello stato. Transizione creata con vecchio valore `"SCONOSCIUTO"`.
- **TC-03** `aggiornaStato()`**:** Aggiornamenti successivi. Le transizioni mantengono traccia del valore precedente corretto.
- **TC-04** `aggiornaStato()`**:** Parametro inesistente. Blocco inserimento: lancia `IllegalArgumentException`.
- **TC-05** `getDispositiviCompatibili()` **(Happy Path):** Comando valido. Restituisce lista contenente se stesso.
- **TC-06** `getDispositiviCompatibili()`**:** Comando non supportato. Restituisce lista vuota.

Gruppo

Il testing ha evidenziato alcune carenze a livello di robustezza ma anche di progettazione, soprattutto per quanto riguarda il comportamento dei metodi `getDispositivi()` e `getDispositiviCompatibili()`, i quali hanno il compito di raccogliere i dispositivi reali all'interno del gruppo. L' implementazione del pattern `Composite` e l'utilizzo della ricorsione non ci rendono la vita semplice, per cui si è reso necessario un refactor dei metodi sopra citati. In particolare sono stati utilizzati dei metodi di supporto che utilizzano dei set per gestire i duplicati ed impedire i loop dovuti alle dipendenze incrociate fra i gruppi (Es. A è sottogruppo di B --- B è sottogruppo di A). Infine è stato inserito un controllo sui target nulli nei metodi `addTarget()` e `removeTarget()`.
Alla luce delle modifiche effettuate si sono delineati i seguenti tests:

- **TC-01** `add/removeTarget()` **(Happy path):** Verifica aggiunta/rimozione targets.
- **TC-02** `add/removeTarget()`**:** Il Target è null: `IllegalArgumentException` deve essere lanciata.
- **TC-03** `addTarget()`**:** Controllo Anti-Loop: Aggiunta gruppo a se stesso. L'inserimento del gruppo deve essere impedito.
- **TC-04** `getDispositivi()`**:** Test ricorsione: Devono essere restituiti tutti e soli i dispositivi reali senza duplicati né gruppi.
- **TC-05** `getDispositivi()`**:** Controllo Anti-Loop: La dipendenza incrociata non deve produrre loop infiniti.
- **TC-06** `getDispositiviCompatibili()`**:** Test compatibilità: L'esito del controllo di compatibilità del comando deve essere in accordo con la definizione del dispositivo e del suo stato in particolare.

ParamStato

- **TC-01 (Happy path):** Test costruttore. Sia il nome del parametro che il suo valore sono validi. Viene creato un oggetto `ParamStato` con nome e valore corrispondenti.
- **TC-02/03 :** Test costruttore. Nome nullo o vuoto. Una `IllegalArgumentException` deve essere lanciata.
- **TC-04** `setValore()`**:** Il vecchio valore deve essere sovrascritto.

DescParametro

- **TC-01** `isValoreValido()`**:** Verifica che il blocco iniziale respinga qualsiasi comando se `readOnly` è `true`.
- **TC-02** `isValoreValido()`**:** Verifica che il valore appartenga alla lista dei valori accettabili dal parametro.
- **TC-03** `isValoreValido()`**:** Verifica che il valore appartenga al range di valori accettabili dal parametro.
- **TC-04** `isValoreValido()`**:** Verifica il formato numerico del valore.

DescDispositivo

- **TC-01:** Test costruttore. Verifica che la descrizione dei parametri non sia nulla o vuota.
- **TC-02:** Verifica comportamento copia difensiva. L'istanza originale della mappa rappresentante la descrizione dello stato non deve essere condivisa. Viene restituita una copia. A fronte di modifiche alla copia la descrizione dello stato interna rimane inalterata.

- **TC-03** `isCompatibile()` **:** Verifica che comandi nulli o inesistenti vengano respinti.
- **TC-04** `isCompatibile()` **:** Verifica che i comandi compatibili vengano accettati e quelli incompatibili scartati.

Comando

É Stato inserito un controllo nel costruttore per evitare che vengano istanziato comandi non validi.

- **TC-01 (Happy path):** Test costruttore.
- **TC-02:** Test costruttore. Comando nullo, vuoto o valore comando nullo. Una `IllegalArgumentException` deve essere lanciata.

Evento

- **TC-01 (Happy path):** Test costruttore. Evento creato correttamente.
- **TC-02:** Test costruttore. Qualcuno dei parametri è `null`. Una `IllegalArgumentException` deve essere lanciata.
- **TC-03:** Verifica comportamento copia difensiva. L'istanza originale della mappa rappresentante le transizioni di stato non deve essere condivisa. Viene restituita una copia.

Cronologia

- **TC-01:** Test stato iniziale. La cronologia deve essere inizialmente vuota.
- **TC-02** `addEvento()` **:** Verifica che gli eventi vengano aggiunti correttamente. Ignora i `null`.
- **TC-03:** Verifica comportamento copia difensiva. L'istanza originale della lista rappresentante la cronologia non deve essere condivisa. Viene restituita una copia.
- **TC-04** `svuotaCronologia()` **:** Verifica che la cronologia venga svuotata correttamente.

Test di Integrazione (UC6 - Esecuzione Comando)

Per effettuare i test di integrazione si è seguita una strategia bottom-up si è partiti dalle classi che implementano le descrizioni dei dispositivi e si è andati avanti ad integrare verso l'alto includendo tutte le classi coinvolte nel flusso di comando dei dispositivi. In particolare sono stati effettuati i seguenti test di integrazione

Descrizioni Integration Test

Questo è il livello più basso dell'integrazione. Verifica la comunicazione tra il `Comando` in ingresso e le definizioni strutturali del dispositivo (`DescDispositivo` e `DescParametro`). In particolare testa la validazione del comando secondo i metodi specificati nella descrizione del dispositivo.

Dispositivo Integration Test

Viene testata l'integrazione tra la classe `Dispositivo` e la sua `DescDispositivo` e nella fattispecie la chiamata al metodo `getDispositiviCompatibili()`.

Gruppo Integration Test

Questo livello testa l'integrazione dei dispositivi fisici all'interno di un `Gruppo`. L'obiettivo è verificare che la richiesta sia propagata a tutti e soli i figli compatibili.

Controller Dispositivo

Infine uniamo i componenti di dominio precedentemente validati e li orchestriamo attraverso il `ControllerDispositivo`. Per garantire l'isolamento del test da fattori esterni, viene utilizzato un servizio di rete fittizio che simula la risposta della rete. L'obiettivo è certificare che l'intero ciclo di vita di una richiesta funzioni alla perfezione, dall'input dell'utente fino alla registrazione nella cronologia.

- **TC-01:** ID target inesistente nel registro. Lancio eccezione `IllegalArgumentException`
- **TC-02:** Comando incompatibile col dispositivo. Comando annullato. Nessuna transizione. Cronologia vuota.
- **TC-03 (Happy Path):** Comando compatibile e target online. Lista transizioni non vuota. Evento salvato in cronologia.